3 Notasi Algoritma

3

Notasi Algoritma

Coba renungkan

Di bab sebelumnya, kita sudah berhasil memahami cara kerja algoritma Searching dan Sorting melalui imajinasi dan analogi mulai dari menebak halaman buku, mencari tumpukan data, hingga menyusun barisan siswa. Ide-ide penyelesaian masalah tersebut terasa sangat logis dan masuk akal saat masih berada di dalam kepala kita.

Namun, pernah kepikiran nggak, bagaimana caranya kita menyampaikan ide-ide cemerlang di otak kita tersebut kepada mesin komputer?

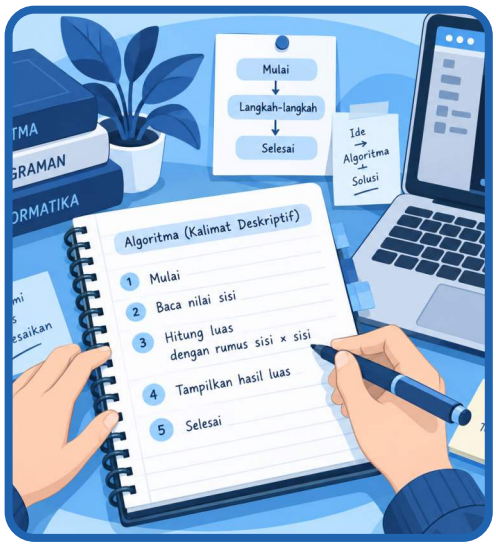
Komputer itu ibarat robot pekerja yang sangat patuh dan bekerja super cepat, tapi sayangnya ia sangat kaku. Komputer tidak paham bahasa manusia sehari-hari, tidak mengerti instruksi yang bertele-tele, dan tidak bisa menebak maksud perasaan kita. Jika kamu langsung mengetikkan instruksi seperti, "Komputer, tolong urutkan data nilai ini dari yang terkecil dong, cepetan ya!", komputer hanya akan error dan bingung.

Agar ide brilian di kepala kita bisa dipahami oleh mesin komputer, kita membutuhkan sebuah "jembatan komunikasi". Jembatan inilah yang dalam ilmu komputer disebut sebagai Notasi Algoritma. Algoritma sendiri adalah suatu kumpulan instruksi terstruktur dan terbatas yang dapat diimplementasikan dalam bentuk program komputer untuk menyelesaikan suatu permasalahan komputasi tertentu. Agar kelak dapat diterjemahkan menjadi kode program, algoritma harus ditulis dalam bentuk yang terdefinisi dengan baik (well-defined) dengan jumlah langkah yang terbatas.

Tiga Tingkatan Notasi Algoritma

Sebuah algoritma yang baik haruslah dapat dikomunikasikan dengan jelas, baik kepada sesama manusia pembangun sistem, maupun kepada mesin yang akan mengeksekusinya. Dalam ilmu komputer, algoritma dapat diekspresikan ke dalam tiga tingkatan format notasi, mulai dari yang paling kasual hingga yang paling mendekati bahasa mesin.

Kalimat Deskriptif (Notasi Alami)



Tingkatan yang paling dasar dan paling bersahabat bagi pemula adalah penulisan menggunakan kalimat deskriptif. Penulisan algoritma dengan cara ini sering juga disebut sebagai notasi alami karena murni menggunakan untaian kalimat bahasa manusia sehari-hari, baik itu menggunakan Bahasa Indonesia maupun Bahasa Inggris. Pada notasi alami ini, tidak ada aturan baku yang kaku mengikat penulisnya. Kalian bebas berekspresi menggunakan berbagai kata kerja operasional yang lazim digunakan dalam percakapan atau instruksi matematis, seperti kata mulai, tulis, baca, tampilkan, jika, maka, dan ulangi. Pendekatan naratif ini sangat cocok dan efektif diterapkan apabila algoritma yang sedang dirancang masih tergolong pendek dan sederhana.

Meskipun menawarkan kebebasan dan kepraktisan yang tinggi, notasi kalimat deskriptif menyimpan sebuah kelemahan mendasar. Jika sebuah program memiliki alur logika yang sangat panjang, rumit, dan kompleks, penulisan instruksi dengan kalimat panjang justru menjadi kurang efektif. Bahasa manusia memiliki sifat dasar yang terkadang ambigu atau mengandung makna ganda. Kebebasan merangkai kata dalam notasi ini dapat memicu kesalahpahaman saat orang lain membaca algoritma tersebut.

Oleh karena itu, agar tetap rapi, terarah, dan terhindar dari ambiguitas, penulisan notasi kalimat deskriptif terbagi ke dalam tiga bagian utama, yaitu judul, deklarasi, dan deskripsi. Bagian judul merupakan bagian yang terdiri dari nama algoritma dan penjelasan singkat tentang algoritma yang akan dibuat. Bagian deklarasi mendefinisikan semua nama yang kelak digunakan pada algoritma, seperti variabel, tipe data, fungsi, hingga konstanta. Terakhir, bagian deskripsi mendefinisikan uraian langkah-langkah sistematis untuk penyelesaian masalah.

Kata Kunci Notasi Alami (kalimat Deskriptif)

Walaupun notasi alami tidak memiliki aturan baku dan menggunakan bahasa manusia sehari-hari, penulisan algoritma yang baik dan sistematis biasanya selalu memuat kata kerja operasional

Sebagai contoh nyata penerapannya, mari kita perhatikan penulisan asli dari algoritma kalimat deskriptif di bawah ini. Terdapat dua contoh, yaitu algoritma sederhana untuk menghitung perkalian dua bilangan bulat dan algoritma logika operasional saat seseorang melakukan penarikan tunai di mesin ATM.

Contoh 1 (Algoritma Perkalian)

```
Judul:
Algoritma Perkalian
(perkalian 2 bilangan bulat. Algoritma memasukkan 2
bilangan bulat, kemudian menghitung hasil perkalian dari
kedua angka tersebut, dan mencetak hasil perkalian
tersebut).

Deklarasi:
bil1, bil2, hasilkali = int

Deskripsi:
Input bil1, bil2
hitung hasilkali = bil1 x bil2
Tampilkan hasilkali
Selesai
```

Ketika membaca Contoh 1 di atas, kalian mungkin bertanya-tanya saat melihat bagian Deklarasi: apa sebenarnya maksud dari tulisan bil1, bil2, hasilkali = int? Bayangkan kalian sedang bersiap memasak di dapur. Sebelum mulai mengolah bahan, kalian tentu harus menyiapkan mangkuk, piring, atau panci terlebih dahulu, bukan? Nah, di dunia komputer, wadah penampung data itu disebut sebagai Variabel. Agar mesin tidak bingung saat memproses instruksi, setiap wadah harus diberi label nama yang jelas, misalnya wadah yang diberi label nama bil1, bil2, dan hasilkali.

Selain diberi nama, wadah tersebut juga harus diberi aturan pasti mengenai jenis isinya, yang disebut sebagai Tipe Data. Kata int yang kalian lihat pada bagian deklarasi tersebut adalah singkatan dari Integer atau bilangan bulat. Artinya, kita sedang memberikan instruksi kepada komputer bahwa wadah-wadah tersebut hanya boleh diisi dengan angka bulat dan tidak boleh diisi dengan teks, kalimat, atau angka pecahan. Konsep mendalam tentang variabel dan tipe data ini akan kita kupas

tuntas pada bab selanjutnya. Jadi untuk saat ini, cukup bayangkan bagian Deklarasi sebagai tahap persiapan untuk menyediakan wadah kosong beserta labelnya sebelum program mulai bekerja.

Sekarang, mari kita lihat penerapan notasi alami ini pada contoh kedua yang logikanya lebih sering kita temui dalam kegiatan sehari-hari.

Contoh 2 (Algoritma TarikTunaiATM)

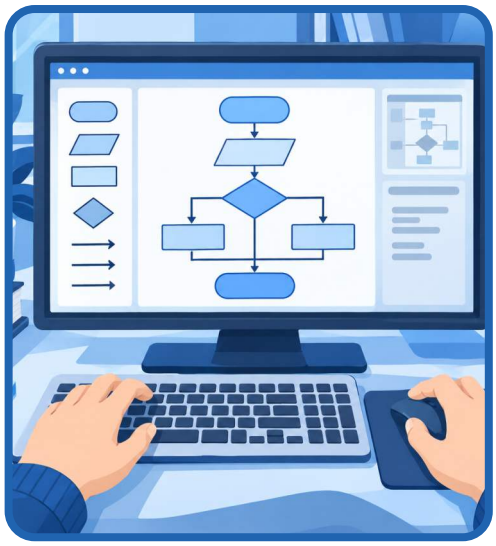
```
Judul:
Algoritma TarikTunaiATM (melakukan penarikan uang tunai
dari mesin ATM. Algoritma akan meminta nominal penarikan,
mengecek apakah saldo awal mencukupi, dan mengeluarkan
uang serta sisa saldo jika syarat terpenuhi).

Deklarasi:
saldoAwal, nominalTarik, sisaSaldo = int

Deskripsi:
Input nominalTarik
Jika saldoAwal lebih besar atau sama dengan nominalTarik
maka:
Hitung sisaSaldo = saldoAwal - nominalTarik
Tampilkan pesan "Penarikan Berhasil"
Tampilkan sisaSaldo
Jika tidak memenuhi syarat maka:
Tampilkan pesan "Maaf, Saldo Anda Tidak Mencukupi"
Selesai
```

Pada Contoh 2 di atas, kita melihat sebuah instruksi logika baru yang sangat penting dalam dunia pemrograman, yaitu Pengambilan Keputusan atau penyeleksian kondisi. Coba perhatikan kalimat yang diawali dengan kata Jika. Mesin ATM di dunia nyata tidak akan asal mengeluarkan uang tunai begitu saja saat tombol ditekan oleh nasabah. Komputer di dalam mesin tersebut telah diprogram untuk mengecek sebuah syarat terlebih dahulu sebelum bertindak.

Apabila syarat pertama terpenuhi (jumlah saldo awal mencukupi untuk ditarik), maka mesin akan mengeksekusi perhitungan matematika untuk mengurangi saldo dan langsung menampilkan pesan keberhasilan. Namun, bagaimana jika

saldo di tabungan ternyata lebih sedikit dari nominal yang ingin ditarik? Komputer tidak boleh diam saja, macet, atau bahkan error. Di sinilah instruksi Jika tidak memenuhi syarat maka: mengambil alih kendali. Instruksi penutup ini bertindak layaknya sebuah rencana cadangan yang akan otomatis dijalankan oleh mesin untuk menolak penarikan dan menampilkan pesan peringatan secara sopan. Melalui contoh mesin ATM ini, kalian bisa melihat bahwa algoritma bukan sekadar urutan alat hitung biasa, melainkan cara kita mengajari komputer agar bisa mengambil keputusan dan merespons situasi yang berbeda secara tepat.

Notasi Flowchart(Diagram Alir)



Jika sebelumnya kalian menggunakan untaian kalimat untuk menuliskan algoritma, cara kedua ini akan mengajak kalian menggunakan pendekatan visual. Flowchart (atau diagram alir) merupakan standar pemodelan yang berfungsi untuk menggambarkan alur logika dari algoritma kalian. Dalam flowchart, urutan operasi atau langkah-langkah penyelesaian masalah direpresentasikan dalam bentuk diagram yang mengalir, mulai dari titik awal (start) hingga mencapai hasil akhir.

Penggunaan flowchart akan membuat algoritma program yang tadinya rumit menjadi jauh lebih mudah untuk dimengerti. Mengapa demikian? Karena instruksi-instruksi di dalamnya tidak lagi berupa paragraf kalimat, melainkan diterjemahkan ke dalam simbol-simbol gambar yang sudah terstandar. Tujuan utamanya sangatlah jelas: membantu kalian memvisualisasikan tahapan penyelesaian masalah secara sederhana, ringkas, dan terarah, dengan menggunakan simbol-simbol yang sangat mudah dipahami.

Pentingnya Penggunaan Flowchart

Penggunaan diagram visual ini tidak hanya memudahkan kalian belajar, tetapi juga memegang peranan yang sangat vital di dunia kerja nyata. Keberadaan flowchart selalu digunakan sebagai acuan utama oleh para programmer untuk mengembangkan sebuah program komputer. Penggunaannya menjadi sangat penting karena beberapa alasan utama berikut:

1 Memperjelas proses yang kompleks

Sebuah program yang rumit pasti akan sulit dipahami jika hanya dijelaskan dengan teks biasa. Pengembang aplikasi sangat membutuhkan flowchart untuk berkomunikasi dengan pelanggannya. Melalui diagram ini, kerumitan sistem akan menjadi lebih sederhana sehingga pelanggan yang masih awam pun bisa lebih mudah menerima penjelasan algoritma yang disampaikan.

2 Meningkatkan efisiensi kerja

Diagram alir dapat digunakan untuk meninjau ulang bagaimana alur kerja sebuah program. Jika dirasa ada celah, program tersebut masih dapat diperbaiki atau dioptimalkan dengan cara menyederhanakan proses maupun mengalihkan alurnya melalui flowchart. Dengan demikian, diagram ini memberikan efisiensi yang tinggi dalam perbaikan sebuah sistem.

3 Memperlihatkan proses yang sedang dijalankan

Dengan menggunakan flowchart, pengembang maupun pelanggan dapat melihat dengan pasti bagaimana sebuah proses berjalan ketika program mulai diaktifkan. Hal ini dimungkinkan karena simbol-simbol di dalamnya memberikan keterangan yang sangat jelas dan detail mengenai apa yang sedang terjadi.

4 Menjadi acuan dasar dalam proses pengembangan

Sebelum mulai mengetik barisan kode (coding), tim pengembang pasti akan merancang flowchart terlebih dahulu. Notasi visual ini digunakan sebagai fondasi dalam perancangan program. Dalam penerapannya, flowchart membantu pengembang untuk lebih mudah memahami alur algoritma dari program yang akan dibangun sehingga proses coding menjadi jauh lebih terarah.

Kata Kunci Notasi Flowchart (Diagram Alir)

Sama halnya dengan nòtasi alami yang sangat bergantung pada kata kerja operasional, perancangan algoritma menggunakan diagram alir mengharuskan kalian untuk menerjemahkan kata kerja tersebut (seperti mulai, masukkan, hitung, jika, dan cetak) langsung ke dalam bentuk simbol bangun datar yang sudah memiliki standar baku

Simbol Flowchart

Bentuk Simbol	Nama	Deskripsi
	Garis alir *(flowline)*	Arah yang menunjukkan aliran program dari awal hingga akhir.
	Terminator	Titik awal atau titik akhir suatu program.
	Proses	Suatu kegiatan komputasi yang dilakukan oleh program: misalnya operasi aritmatika.
	Keputusan	Merupakan titik percabangan yang salah satu cabangnya dapat dilalui oleh program berdasarkan suatu kondisi.
	Masukan (*Input*)/ Keluaran (*Output*)	Melambangkan titik saat program akan menerima suatu data atau menghasilkan suatu informasi.
	Subprogram	Melambangkan suatu kegiatan atau proses lain yang telah didefinisikan sebelumnya.
	Penghubung dalam Halaman	Digunakan untuk menghubungkan suatu titik pada diagram alir ke titik lain pada halaman yang sama.
	Penghubung antarhalaman	Digunakan untuk menghubungkan suatu titik pada diagram alir ke titik lain pada halaman yang berbeda. Digunakan apabila diagram lain cukup kompleks sehingga tidak dapat digambar dalam satu halaman.

Implementasi Flowcart

1 Diagram Alir 1: Menghitung Luas Persegi

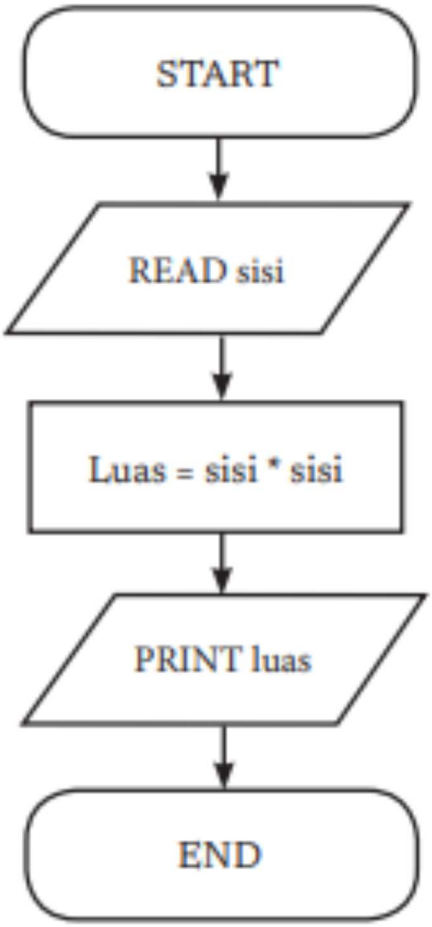


Diagram alir dibaca mulai dari simbol START, lalu mengikuti arah panah. Untuk menghitung luas persegi, kalian memerlukan sebuah data, yaitu panjang sisi. Panjang sisi ini dibaca pada diagram alir dengan menggunakan kata kunci READ dan disimpan dalam sebuah variabel bernama sisi. Setelah itu, kalian melakukan suatu proses ekspresi matematika untuk menghitungluas persegi menggunakan rumus yang telah kalian ketahui, yaitu luas = sisi x sisi. Hasil perhitungan tersebut disimpan pada sebuah variabel bernama luas. Walaupun telah memperoleh jawaban yang dicari, komputer perlu di instruksikan secara spesifik untuk mengeluarkan jawaban tersebut. Pada diagram ini, kata kunci PRINT digunakan untuk mencetak nilai dari variabel luas yang telah diperoleh pada tahap sebelumnya. Setelah PRINT, algoritma berakhir karena simbol setelahnya adalah simbol END.

2 Diagram Alir 2: Sistem Login Akun Media Sosial

Setelah mengenal simbol-simbol dasar diagram alir, kalian mungkin berpikir bahwa algoritma hanya digunakan oleh para programmer untuk membuat aplikasi yang rumit. Padahal, tanpa kalian sadari, logika flowchart selalu mengelilingi aktivitas kita setiap hari!

Mari kita ambil satu contoh kegiatan yang pasti sering kalian lakukan: masuk atau login ke akun media sosial. Pernahkah kalian membayangkan apa yang sebenarnya dilakukan oleh mesin di dalam aplikasi tersebut saat kalian mengetikkan username dan password?

Proses login yang hanya memakan waktu sekian detik itu sebenarnya adalah sebuah runtutan algoritma yang sangat disiplin. Coba kita bedah alur diagramnya:

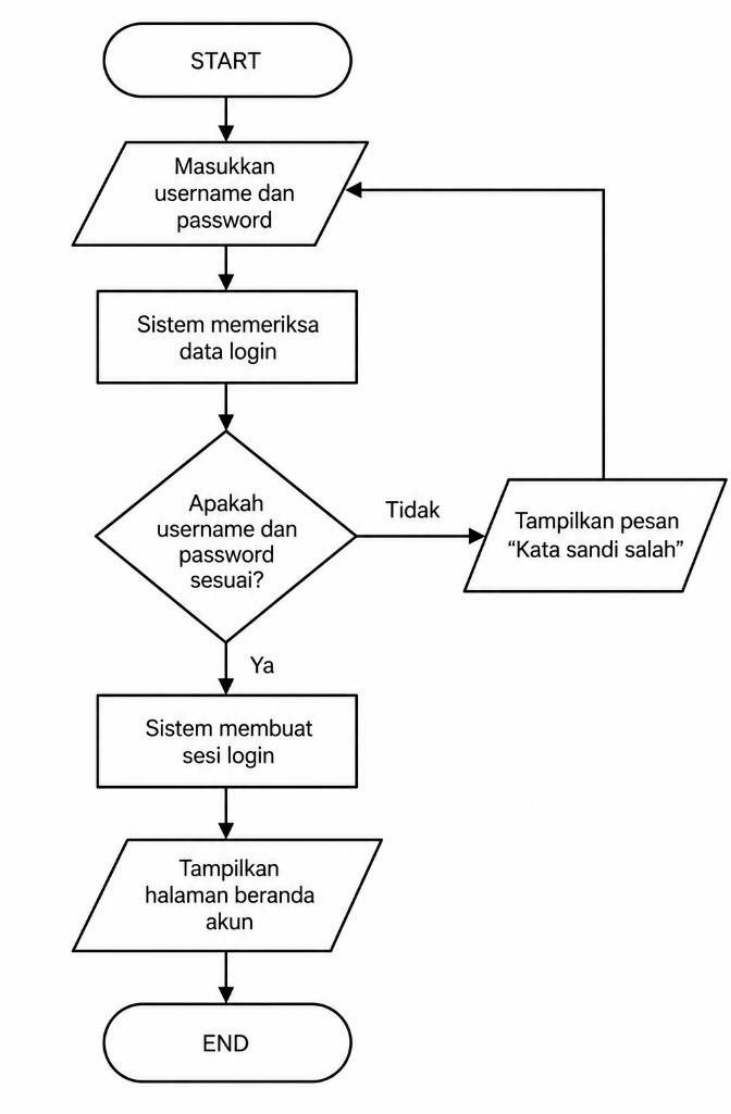


Diagram alir ini dibaca mulai dari simbol START, lalu bergerak mengikuti arah panah ke bawah. Sebagai langkah awal, program memerlukan data dari pengguna. Data berupa username dan password ini dibaca pada diagram alir menggunakan simbol masukan (jajargenjang).Setelah data dimasukkan, sistem akan memproses dan memeriksa kecocokan data tersebut di dalam pangkalan data (database) menggunakan simbol proses (persegi panjang).Selanjutnya, alur tiba pada simbol keputusan (belah ketupat) untuk mengevaluasi suatu kondisi: "Apakah username dan password sesuai?". Jika kondisinya bernilai "Tidak", alur panah akan berbelok menuju simbol keluaran untuk menampilkan pesan peringatan "Kata sandi salah". Setelah itu, garis panah membawa alur berputar kembali ke atas (looping), meminta kalian untuk memasukkan password ulang.Sebaliknya, jika kondisi bernilai "Ya", program akan mengeksekusi proses pembuatan sesi login, lalu bergerak ke simbol keluaran untuk menampilkan halaman beranda akun. Setelah pengguna berhasil masuk ke halaman utama, tugas algoritma dinyatakan selesai dan ditutup dengan simbol END.

Kata Kunci Notasi Flowchart (Diagram Alir)

Saat membaca diagram alir, selalu ikuti arah ujung panah. Berhentilah sejenak pada simbol belah ketupat (Keputusan) untuk mengecek cabang "Ya" dan "Tidak". Jika ada panah yang berbelok kembali ke atas, itu menandakan proses perulangan (looping) sedang terjadi!

Notasi Pseudocode (Kode Semu)

Setelah merangkai logika menggunakan bahasa sehari-hari (Notasi Alami) dan menggambarnya dengan bangun datar (Flowchart), kini saatnya kalian berkenalan dengan cara ketiga: Pseudocode. Secara bahasa, pseudocode berasal dari kata pseudo yang berarti "semu" atau "tiruan", dan code yang berarti "kode program". Sesuai namanya, pseudocode adalah kode program tiruan. Ini bukanlah bahasa pemrograman sungguhan yang bisa langsung dijalankan oleh mesin komputer, melainkan sebuah draf rancangan yang ditulis khusus agar mudah dibaca oleh otak manusia.

Mengapa kita butuh menulis kode semu? Coba bayangkan pseudocode sebagai sebuah jembatan penghubung. Notasi alami terkadang terlalu panjang, bertele-tele, dan maknanya bisa ambigu. Di sisi lain, bahasa pemrograman asli (seperti C, C++, atau Python) memiliki aturan tata bahasa (syntax) yang sangat ketat. Lupa menaruh satu tanda titik koma (;) saja, program bisa langsung rusak atau error!

Nah, pseudocode berada tepat di tengah-tengahnya. Melalui notasi ini, kalian bisa berlatih menyusun algoritma layaknya seorang programmer profesional, namun tanpa perlu takut program kalian akan error karena aturan penulisan yang kaku.

Dalam menyusun pseudocode, kalian akan mulai meninggalkan kalimat panjang dan menggantinya dengan kata kunci (keyword) ringkas yang sering ditemui dalam dunia pemrograman, seperti INPUT, PRINT, atau IF-THEN. Ingat, tidak ada aturan baku internasional yang mengikat penulisan pseudocode. Selama logikanya runtut, jelas, dan programmer lain bisa memahaminya, maka draf pseudocode kalian sudah dianggap benar.

Pentingnya Menggunakan Pseudocode

Walaupun hanya berupa "kode semu" dan tidak bisa langsung dijalankan oleh komputer, pseudocode memegang peran yang sangat krusial. Pada intinya, notasi ini digunakan untuk menuliskan algoritma dengan cara yang lebih sederhana.Berikut adalah beberapa alasan mengapa pseudocode sangat penting bagi kalian yang baru mulai merancang program:

Memberikan Kemudahan Pemahaman

Karena tidak terikat dengan aturan baku (syntax) yang rumit, konsep dan struktur pseudocode jauh lebih mudah dipahami. Kalian bisa lebih fokus memikirkan alur logika yang tepat tanpa takut program menjadi error hanya karena salah ketik

Menjadi Jembatan Penghubung yang Baik

Bagi pemula, pseudocode adalah sarana transisi yang sempurna. Ia mampu menjembatani logika bahasa pikiran manusia agar lebih selaras dengan struktur program komputer yang sedang dirancang

Sebagai Sarana Dokumentasi

Rancangan algoritma yang ditulis menggunakan pseudocode dapat disimpan sebagai catatan rekam jejak. Jika suatu saat kalian ingin meningkatkan atau mengembangkan sistem, kalian bisa membaca pseudocode-nya terlebih dahulu tanpa harus membedah program aslinya yang lebih kompleks.

Penerjemah Flowchart

Ketika alur pada flowchart sudah terlalu panjang dan memakan banyak ruang untuk digambar, programmer menggunakan pseudocode untuk menerjemahkan visual diagram alir tersebut ke dalam bentuk teks secara lebih efisien dan efektif

Memudahkan Proses Pengembangan Sistem

Berkat bahasanya yang sederhana namun terstruktur, menemukan kesalahan logika (bug) atau melakukan modifikasi alur aplikasi jauh lebih cepat dan mudah dilakukan di atas kertas pseudocode dibandingkan merevisi program yang sudah jadi.

Ciri-ciri Pseudocode

Agar kalian bisa membedakan pseudocode dengan bahasa pemrograman asli maupun bahasa sehari-hari, mari kenali beberapa karakteristik utamanya:

1 Berbahasa Sederhana

Pseudocode menggunakan bahasa yang ringkas dan tidak rumit (biasanya mengadaptasi bahasa Inggris dasar). Tujuannya murni agar alurnya mudah dibaca dan dipahami oleh logika manusia, bukan oleh mesin.

2 Bebas Aturan Baku

Berbeda dengan bahasa pemrograman yang memiliki aturan tata bahasa (syntax) ketat, pseudocode tidak memiliki standar baku internasional yang mengikat. Selama logikanya jelas, kalian bebas menuliskannya dengan gaya kalian sendiri tanpa takut error.

3 Meminjam Simbol Pemrograman

Walaupun bebas, pseudocode tetap sering meminjam beberapa simbol operasi matematika atau sintaks dari program asli untuk mempersingkat penulisan. Contohnya, menggunakan tanda panah ← untuk memasukkan nilai, atau tanda perbandingan matematis seperti <, >, <=, dan >=

4 Terstruktur dan Berurutan

Layaknya sebuah resep masakan, pseudocode selalu ditulis dalam bentuk langkah-langkah yang disusun secara berurutan (sekuensial) dari atas ke bawah untuk menyelesaikan suatu permasalahan tertentu.

Kategori Pseudocode

Penggunaan notasi pada pseudocode bertujuan untuk memudahkan kita memahami fungsi dan alur algoritma sebelum benar-benar menyusun kode pemrograman asli. Walaupun tidak memiliki aturan baku yang ketat, para programmer umumnya sepakat menggunakan beberapa kategori notasi standar berikut ini:

- **INPUT (Masukan)**:

  Digunakan ketika program perlu menerima data atau interaksi dari pengguna (misalnya melalui ketikan keyboard atau klik mouse)

  - Contoh notasi: Read, Get, Obtain, Input
- **OUTPUT (Keluaran)**

  Digunakan untuk menampilkan informasi, pesan peringatan, atau hasil akhir dari sebuah proses ke layar pengguna.

  - Contoh notasi: Print, Write, Display, Show
- **COMPUTE (Pemrosesan)**

  Digunakan saat program harus memproses data masukan atau melakukan perhitungan matematika.

  - Contoh notasi: Compute, Calculate, Determine.
- **DECISION (Keputusan/Percabangan)**

  Digunakan untuk menunjukkan adanya persimpangan logika, di mana program harus memilih tindakan berbeda berdasarkan suatu syarat atau kondisi tertentu.

Penggunaan notasi Pseudocode

Tujuan utama penulisan pseudocode adalah menyediakan draf instruksi sistematis yang nantinya akan diterjemahkan (translasi) ke dalam bahasa pemrograman sungguhan, seperti C++, Python, atau Java. Karena tidak terikat aturan baku yang kaku, penulisan notasinya sangat ringkas.Mari kita lihat perbandingannya:

1 Contoh 1: Menampilkan Kalimat

Jika kalian ingin menginstruksikan komputer untuk menampilkan teks ke layar, kalian cukup menulis:

```
WRITE ("Semangat belajar, ya!")
```

Sebagai perbandingan, jika pseudocode tersebut diterjemahkan ke dalam bahasa pemrograman Java, kodenya akan terlihat jauh lebih kompleks dan terikat aturan tanda baca:

```
System.out.print("Semangat belajar, ya!");
```

2 Contoh 2: Memasukkan Nilai

Jika kalian ingin memindahkan atau menyimpan sebuah nilai dari variabel m ke dalam variabel n, notasinya ditulis menggunakan arah panah:

```
n ← m
```

Tanda panah (←) memiliki arti "isikan nilai ke dalam...". Jika diterjemahkan ke dalam bahasa pemrograman asli, tanda panah ini biasanya diubah menjadi tanda sama dengan (=).

Struktur Penulisan Pseudocode

Walaupun penulisannya bebas, pseudocode yang baik umumnya tetap mengikuti tiga struktur utama. Layaknya sebuah karangan tulisan, struktur ini memastikan draf algoritma kalian tetap rapi dan mudah dipahami oleh programmer lain.

a. Bagian Judul (Header)

Berisi nama program dan penjelasan singkat tentang apa yang dilakukan program tersebut. Penulisannya biasa diawali dengan kata PROGRAM diikuti nama program (tanpa spasi), dan penjelasan singkat yang diapit oleh tanda kurung kurawal {}.

b. Bagian Deklarasi

Ini adalah tempat untuk mengidentifikasi dan mengenalkan semua "data" yang akan dipakai di dalam program. Data tersebut bisa berupa variabel, tipe data, atau konstanta tetap. Komputer harus mengenali nama-nama data ini terlebih dahulu sebelum mulai memprosesnya. Jika algoritma kalian sangat sederhana dan tidak membutuhkan penyimpanan data, bagian ini boleh dikosongkan.'

c. Bagian Isi Algoritma

Ini adalah inti dari program. Bagian ini berisi sekumpulan instruksi langkah demi langkah, proses perhitungan, hingga perintah untuk menampilkan hasil akhir.

Untuk melihat bentuk utuhnya, berikut adalah contoh kerangka pseudocode untuk program menghitung luas lingkaran:

```
PROGRAM LuasLingkaran
{Program membaca panjang jari-jari (r), menghitung luas
lingkaran, lalu mencetak hasilnya ke layar

DEKLARASI:
const pi = 3.14 {nilai konstanta tetap}
r : real        {variabel jari-jari berupa angka desimal}
L : real        {variabel luas berupa angka desimal}

ALGORITMA:
READ (r)        {meminta pengguna memasukkan nilai r}
L ← pi * r * r  {proses menghitung luas lingkaran}
WRITE (L)       {menampilkan hasil perhitungan luas ke layar}
```

Kata Kunci: Struktur Pseudocode

Ibarat resep masakan,' pseudocode memiliki 3 bagian: Judul (nama program), Deklarasi (menyiapkan variabel/data), dan Algoritma (langkah penyelesaian). Gunakan juga tanda panah (←) untuk memasukkan nilai ke dalam variabel!

Glosarium

Algoritma:
Urutan langkah-langkah logis dan sistematis yang disusun untuk menyelesaikan suatu masalah atau mencapai tujuan tertentu.

Berpikir Komputasional (Computational Thinking):
Metode pemecahan masalah dengan menerapkan teknik ilmu komputer, seperti dekomposisi, pengenalan pola, abstraksi, dan algoritma.

Deklarasi:
Bagian dalam pseudocode atau program yang berfungsi untuk mengenalkan atau mendefinisikan nama variabel, konstanta, maupun tipe data sebelum digunakan.

Diagram Alir (Flowchart):
Representasi visual dari sebuah algoritma yang menggunakan berbagai simbol bangun datar dengan fungsi standar yang telah disepakati.

Iterasi (Looping):
Proses perulangan suatu langkah atau sekumpulan instruksi di dalam algoritma hingga syarat atau kondisi tertentu terpenuhi.

Keputusan (Decision):
Titik persimpangan logika dalam algoritma (biasanya disimbolkan dengan belah ketupat) di mana program harus memilih tindakan berdasarkan kondisi "Ya" atau "Tidak".

Konstanta:
Nilai data di dalam program yang bersifat tetap dan tidak dapat diubah selama program berjalan (misalnya nilai konstanta pi = 3.14).

Notasi Alami:
Cara penulisan algoritma menggunakan bahasa manusia sehari-hari berupa kalimat deskriptif..

Glosarium

Pseudocode (Kode Semu):
Notasi rancangan algoritma berupa kode tiruan yang menyerupai bahasa pemrograman, namun ditulis menggunakan bahasa manusia agar mudah dibaca dan dipahami.

Sintaks (Syntax):
Aturan baku tata bahasa dalam penulisan bahasa pemrograman asli yang wajib dipatuhi agar program dapat dijalankan tanpa error.

Subprogram:
Bagian atau modul kecil dari program utama yang dirancang secara terpisah untuk menyelesaikan tugas yang lebih spesifik.

Translasi:
Proses menerjemahkan draf algoritma (seperti flowchart atau pseudocode) ke dalam kode bahasa pemrograman yang sesungguhnya.

Variabel:
Tempat penampungan data di dalam memori komputer yang nilainya dapat berubah-ubah selama program tersebut berjalan (misalnya variabel luas atau sisi).